

Solving a 4x4 Maze with MyCobot Pro 600

1. INTRODUCTION

This report is about using the robotic arm MyCobot Pro 600 for solving the 4x4 maze. The experiment demonstrates a robotic arm of precision and adaptability for motion planning. We use the robot and program its way through this maze with an optimal solution, showcasing its potential for problem-solving and realistic applications in robotics.

2. METHOD

Steps to solve 4x4 maze using the mycobot pro 600 :

- **Capturing and Defining Maze:** The 4x4 maze layout is captured using the AI Kit camera. Based on the captured image, the start and end of the maze are defined.
- **Path Planning:** We use the previously developed mapping function to map the maze and create a path-planning algorithm based on the A* algorithm. This gives us an optimum route through the maze that is capable of solving upto a 50x50 maze, the result will be in the form of a series of x,y coordinates which need to be stored in a CSV file.
- **Inverse Kinematics and Digital Twin Simulation:** The MATLAB code reads the CSV file containing x,y coordinates and provides them as an input to the inverse kinematics solver. The solver then determines the joint angles for the robotic arm and moves its digital twin along the maze path. These joint angles are stored in another CSV file.
- **Robot Execution:** Joint angles are received from the CSV file for the MyCobot Pro 600, allowing it to physically follow the path on the maze and thus carry out the task.

Matlab code

```
robotArm = importrobot("pro600.urdf");
robotArm.DataFormat = 'column';
homeConfig = [8.63 -130.20 -128.53 -11.27 90.00 1.38]*pi/180;
endEffector = 'link6';
homeTransform = getTransform(robotArm, homeConfig, endEffector);
homeTransform(1:4,4) = [0;0;0;1];

show(robotArm,homeConfig);
axis auto;
view([230,10]);

toolHomePos = [0.4550 0.0010 0.4340];

[rowData, colData] = csvimport('waypoints.csv', 'columns', {'Row', 'Column'});

[mappedX, mappedY] = mapper(colData, rowData);

mappedZ = 0.0692 * ones(size(mappedX));

trajectoryPoints = double([mappedX mappedY mappedZ]);

orientMatrix = ([0 -pi/2 pi]' .* ones(size(trajectoryPoints)));

% Timing and velocity initialization
trajectoryTimes = 0:2:(2 * (numel(mappedX)-1));
timeStep = 0.25;
fullTimes = 0:timeStep:trajectoryTimes(end);
```

```

velocityProfile = 0.1 * [0 1 0;
    -1 0 0;
    0 -1 0;
    1 0 0;
    0 1 0];

accelerationProfile = zeros(size(velocityProfile));
accelerationTimes = diff(trajecoryTimes)/4;

ikSolver = inverseKinematics('RigidBodyTree',robotArm);
ikSolverWeights = [1 1 1 1 1 1];
ikInitConfig = homeConfig;
ikInitConfig(ikInitConfig > pi) = ikInitConfig(ikInitConfig > pi) - 2*pi;
ikInitConfig(ikInitConfig < -pi) = ikInitConfig(ikInitConfig < -pi) + 2*pi;

plotOption = 1;
show(robotArm,homeConfig,'Frames','off','PreservePlot',false);
hold on
if plotOption == 1
    trajectoryPlot = plot3(trajecoryPoints(1,1),trajecoryPoints(2,1),trajecoryPoints(3,1),'b.-');
end
plot3(trajecoryPoints(1,:),trajecoryPoints(2,:),trajecoryPoints(3,:), 'ro','LineWidth',2);
axis auto;
view([230,10]);

applyOrientation = true;
totalWaypoints = size(trajecoryPoints,2);
numRobotJoints = numel(robotArm.homeConfiguration);
jointConfigs = zeros(numRobotJoints,totalWaypoints);
for idx = 1:totalWaypoints
    if applyOrientation
        targetPose = trvec2tform(trajecoryPoints(:,idx)) * eul2tform(orientMatrix(:,idx));
    else
        targetPose = trvec2tform(trajecoryPoints(:,idx)) * eul2tform([pi/2 0 pi/2]); %#ok<UNRCH>
    end
    [configResult,~] = ikSolver(endEffector,targetPose,ikSolverWeights,ikInitConfig);
    jointConfigs(:,idx) = configResult;
end

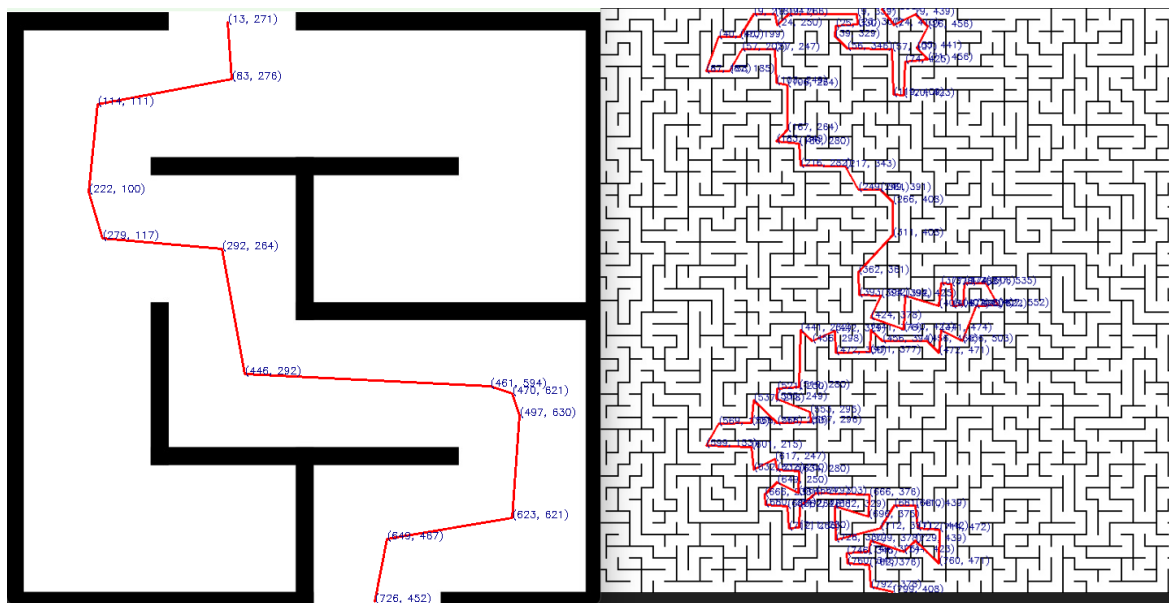
trajectoryType = 'trap';
switch trajectoryType
    case 'trap'
        [jointPos,jointVel,jointAccel] = trapveltraj(jointConfigs,numel(fullTimes), ...
            'AccelTime',repmat(accelerationTimes,[numRobotJoints 1]), ...
            'EndTime',repmat(diff(trajecoryTimes),[numRobotJoints 1]));
    case 'cubic'
        [jointPos,jointVel,jointAccel] = cubicpolytraj(jointConfigs,trajecoryTimes,fullTimes, ...
            'VelocityBoundaryCondition',zeros(numRobotJoints,totalWaypoints));
    case 'quintic'
        [jointPos,jointVel,jointAccel] = quinticpolytraj(jointConfigs,trajecoryTimes,fullTimes, ...
            'VelocityBoundaryCondition',zeros(numRobotJoints,totalWaypoints), ...
            'AccelerationBoundaryCondition',zeros(numRobotJoints,totalWaypoints));
    case 'bspline'
        ctrlPoints = jointConfigs;
        [jointPos,jointVel,jointAccel] = bsplinepolytraj(ctrlPoints,trajecoryTimes([1 end]),fullTimes);
        jointVel(:,1) = zeros(7,1);
    otherwise
        error('Invalid trajectory type! Use "trap", "cubic", "quintic", or "bspline"');
end

for idx = 1:numel(fullTimes)

    currentConfig = jointPos(:,idx);

    eeTransform = getTransform(robotArm,currentConfig,endEffector);
    if plotOption == 1

```



Python code

Python code to solve 4x4 maze

```

import csv
import cv2
import numpy as np
import heapq
import math

coords = []

def mouse_handler(evt, pos_x, pos_y, *args):
    global coords
    if evt == cv2.EVENT_LBUTTONDOWN:
        coords.append((pos_y, pos_x))

def binarize_image(img):
    grayscale = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    _, binary_img = cv2.threshold(grayscale, 127, 255, cv2.THRESH_BINARY)
    return binary_img

def compute_cost_array(bin_img):
    dist_map = cv2.distanceTransform(bin_img, cv2.DIST_L2, 5)
    inv_cost = np.max(dist_map) - dist_map
    inv_cost[bin_img == 0] = np.inf
    return inv_cost

def estimate_dist(loc_a, loc_b):
    return abs(loc_a[0] - loc_b[0]) + abs(loc_a[1] - loc_b[1])

def a_star_path(bin_img, cost_map, start, end):
    if bin_img[start] == 0 or bin_img[end] == 0:
        raise ValueError("Start or end point is invalid.")

    h, w = bin_img.shape
    open_heap = []
    heapq.heappush(open_heap, (0 + estimate_dist(start, end), 0, start))
    path_tree = {}
    g_score = {start: 0}
    f_score = {start: estimate_dist(start, end)}
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while open_heap:
        _, current_g, current = heapq.heappop(open_heap)
        if current == end:
            solution = []
            while current in path_tree:
                solution.append(current)
                current = path_tree[current]
            solution.append(start)
            return solution[::-1]

        for move in moves:
            neighbor = (current[0] + move[0], current[1] + move[1])
            if not (0 <= neighbor[0] < h and 0 <= neighbor[1] < w):
                continue
            if bin_img[neighbor] == 0:
                continue

            new_cost = current_g + cost_map[neighbor]
            if new_cost < g_score.get(neighbor, float('inf')):
                path_tree[neighbor] = current
                g_score[neighbor] = new_cost
                f_score[neighbor] = new_cost + estimate_dist(neighbor, end)
                heapq.heappush(open_heap, (f_score[neighbor], new_cost, neighbor))

    raise ValueError("No valid path found.")

```

```

def streamline_path(route, threshold=5):
    def dist_to_line(pt, line_start, line_end):
        if line_start == line_end:
            return math.sqrt((pt[0] - line_start[0])**2 + (pt[1] - line_start[1])**2)
        numerator = abs((line_end[1] - line_start[1]) * pt[0] - (line_end[0] - line_start[0]) * pt[1] +
            line_end[0] * line_start[1] - line_end[1] * line_start[0])
        denominator = math.sqrt((line_end[1] - line_start[1])**2 + (line_end[0] - line_start[0])**2)
        return numerator / denominator

    def simplify(route, threshold):
        if len(route) < 3:
            return route
        start, end = route[0], route[-1]
        max_dist, idx = 0, 0
        for i in range(1, len(route) - 1):
            dist = dist_to_line(route[i], start, end)
            if dist > max_dist:
                idx, max_dist = i, dist
        if max_dist > threshold:
            left = simplify(route[:idx + 1], threshold)
            right = simplify(route[idx:], threshold)
            return left[:-1] + right
        return [start, end]

    return simplify(route, threshold)

def draw_path(img, route):
    annotated_img = img.copy()
    for i in range(len(route) - 1):
        start = (route[i][1], route[i][0])
        end = (route[i + 1][1], route[i + 1][0])
        cv2.line(annotated_img, start, end, (0, 0, 255), 2)
        cv2.putText(annotated_img, f"{route[i]}", start, cv2.FONT_HERSHEY_SIMPLEX, 0.4, (139, 0, 0), 1)

    last_point = (route[-1][1], route[-1][0])
    cv2.putText(annotated_img, f"{route[-1]}", last_point, cv2.FONT_HERSHEY_SIMPLEX, 0.4, (139, 0, 0), 1)
    return annotated_img

def write_to_csv(route, file_path):
    with open(file_path, mode='w', newline='') as out_file:
        writer = csv.writer(out_file)
        writer.writerow(["Row", "Column"])
        writer.writerows(route)

def process_maze(csv_path):
    global coords

    cap = cv2.VideoCapture(0)
    if not cap.isOpened():
        raise RuntimeError("Camera not accessible.")

    while True:
        ret, frame = cap.read()
        if not ret:
            raise RuntimeError("Failed to capture frame.")

        cv2.imshow("Camera Feed", frame)
        key = cv2.waitKey(1)
        if key == ord('s'):
            maze_img = frame.copy()
            break
        elif key == ord('q'):
            cap.release()
            cv2.destroyAllWindows()
            return

    cap.release()
    cv2.destroyAllWindows()

    maze_bin = binarize_image(maze_img)
    maze_costs = compute_cost_array(maze_bin)

    cv2.imshow("Select Points", maze_img)
    cv2.setMouseCallback("Select Points", mouse_handler)
    cv2.waitKey(0)

```

```

cv2.destroyAllWindows()

if len(coords) != 2:
    raise ValueError("Two points required.")

start, end = coords
path = a_star_path(maze_bin, maze_costs, start, end)
smoothed_path = streamline_path(path, threshold=10)

write_to_csv(smoothed_path, csv_path)

solved_img = draw_path(maze_img, smoothed_path)
cv2.imshow("Solved Maze", solved_img)
cv2.waitKey(0)
cv2.destroyAllWindows()

return smoothed_path

output_path = r"C:\Users\harsh\Desktop\Robotics Lab\FinalProject\Lab4\Lab4\waypoints.csv"
final_path = process_maze(output_path)
print(f"Path saved to {output_path}")

```

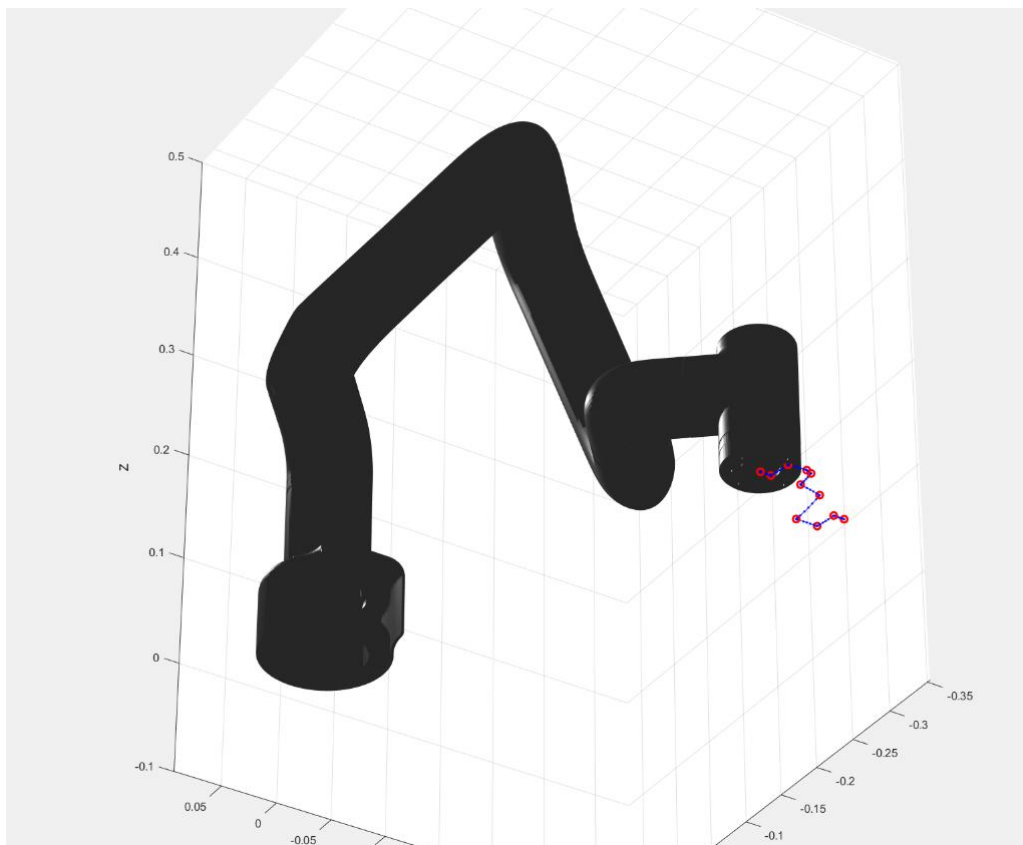


Fig 2.2 Digital twin of mycobot pro 600 following maze path

3. RESULTS AND DISCUSSION

The AI Kit camera successfully captured and mapped the 4x4 maze. The A* algorithm generated an optimal path and efficiently output a series of x, y coordinates, which were stored in a CSV file. These coordinates were then processed by the MATLAB inverse kinematics solver to compute the joint angles, which were further validated through a digital twin simulation. These angles were then fed into the MyCobot Pro 600 to navigate it physically through the maze and accurately replicate the planned path. This project demonstrates how effectively path planning combined with inverse kinematics and robotic execution works for spatial tasks. Future work should focus on real-time adaptability and scalability to even larger mazes.

4. CONCLUSION

This project has well integrated AI-based path planning, inverse kinematics, and robotic execution to solve the 4x4 maze using MyCobot Pro 600. An A* algorithm successfully generates the most optimal path, whereas MATLAB's inverse kinematics solver calculates the right angles required at each joint. Further, this digital twin simulation ascertains reliability before the real execution of navigating the maze with a robotic arm in precision. This approach showcases the potential of advanced algorithms combined with robotics in solving spatial problems. Future work should focus on enhancing real-time adaptability and scalability to solve larger and more complex mazes.
